

Universidad Tecnica Estatal de Quevedo

Facultad de Ciencias de la Computacion y Diseno Digital | Carrera de Ingenieria de Software (Redisenio)

Aplicaciones Web

Quinto nivel | Periodo Academico Presencial 2026-2027 (PPA)

Rubrica de evaluacion — Examen de la Unidad II

Herramientas de programacion web del lado del servidor (PERL, PHP, Java, ASP.NET, seguridad)

Resultado de aprendizaje evaluado: el estudiante *construye una aplicacion web dinamica del lado del servidor que integra el ciclo request-response de HTTP, un mecanismo de autenticacion basado en sesion, el acceso a datos con SQL parametrizado, la estructura por capas y las defensas basicas del OWASP Top 10:2021 [4] en una de las tres pilas trabajadas en la Unidad II (PHP 8.3+PDO [2], Java con Spring Boot 3 [5] o ASP.NET Core 8 [1]).*

Aporte al perfil de egreso. La Unidad II es la segunda de las cuatro que componen la asignatura y constituye la puerta de entrada al desarrollo profesional del lado del servidor. Su dominio es prerrequisito directo del Proyecto Fin de Curso (PFC) y de las Unidades III (persistencia y patrones) y IV (MVC y servicios).

1. Estructura del examen

El examen consta de dos partes complementarias cuya suma constituye la calificacion total de la Unidad II.

Parte	Modalidad	Peso
Parte teorica	Cuestionario de seleccion multiple de 20 preguntas sobre los contenidos de la Unidad II. Todas las alternativas de cada pregunta corresponden a terminos, versiones o valores presentes en el material del curso.	40 %
Parte practica	Resolucion del problema planteado en la Practica Unidad II — micro CRUD web seguro con autenticacion, en la pila del servidor que el estudiante elija (PHP 8.3, Spring Boot 3 o ASP.NET Core 8).	60 %
Total		100 %

2. Directrices habilitantes de la evaluacion

Las siguientes tres condiciones son **habilitantes**: la ausencia de cualquiera de ellas hace que el examen no sea evaluable y la nota resultante sea **cero** en la parte practica.

- Publicacion en el SGA dentro del plazo (habilitante).** El estudiante sube al SGA una unica pagina PDF con: (a) apellidos y nombres completos, (b) numero de cedula, (c) paralelo, (d) fecha del examen, (e) pila tecnologica elegida, (f) enlace HTTPS al repositorio GitHub publico y (g) el hash abreviado (7 caracteres) del ultimo commit anterior o igual a las 10H45.
- Repositorio verificable en GitHub (habilitante).** El repositorio debe ser **publico**, el enlace debe abrir directamente el repositorio (no un fork privado ni un enlace roto), y la autoria de los commits debe corresponder al estudiante. La estructura del repositorio debe permitir la **clonacion, construccion y ejecucion completa** de la aplicacion desde una maquina limpia siguiendo estrictamente el **README.md**, sin conocimiento previo del proyecto.
- Corte del historial Git a las 10H45 (habilitante).** El *ultimo commit* del repositorio debe tener **AuthorDate** y **CommitDate** en zona *America/Guayaquil* (UTC-5) **a mas tardar a las 10H45** del dia del examen. Los commits posteriores no se consideran para la calificacion. Verificacion:

```
git log --pretty=format:'%h | %an | %ad | %s' --date=iso-strict | head -30
git rev-list -1 --before='YYYY-MM-DD 10:45:00 -0500' HEAD
```

3. Parte teorica — Cuestionario de seleccion multiple (40 %)

Criterio de calificacion. Cuestionario de 20 preguntas de seleccion multiple de igual valor sobre los contenidos de la Unidad II: historia y sintaxis de PERL y PHP, servlets y Spring Boot, ASP.NET Core, gestion de sesion, hash de contraseñas, defensas contra SQL Injection, XSS y CSRF, y cabeceras de seguridad HTTP.

La calificacion es proporcional al numero de respuestas correctas, sin penalizacion por respuestas incorrectas:

$$\text{Nota teorica} = \frac{\text{respuestas correctas}}{20} \times 40 \%$$

4. Parte practica — Planteamiento del problema (60 %)

4.1 Contexto

La Facultad de Ciencias de la Computacion y Diseno Digital requiere un micro-sistema web para la administracion de las *materias* que ofrece la carrera de Ingenieria de Software. El sistema **debe estar protegido por autenticacion**: solo un usuario administrador previamente registrado accede al mantenimiento. El estudiante desarrolla el sistema en una unica pila tecnologica del lado del servidor y lo entrega en un repositorio GitHub verificable, autocontenido y ejecutable con un solo comando desde una clonacion limpia.

4.2 Entidad de dominio: Materia

Atributo	Tipo	Restricciones
id	entero (autonumerico)	clave primaria.
codigo	cadena de 6 caracteres	unico, obligatorio (p. ej. APW501).
nombre	cadena 5–80 caracteres	obligatorio.
creditos	entero 1–6	obligatorio, con CHECK en BD.
semestre	entero 1–10	obligatorio, con CHECK en BD.
activa	booleano	valor por defecto <code>true</code> ; la eliminacion es <i>logica</i> .

4.3 Usuario administrador semilla (obligatorio)

El esquema debe crear un usuario `admin` con contrasena `Admin*2026` **almacenada como hash BCrypt** [4, 3]. Este usuario debe existir al arrancar la aplicacion desde una BD vacia (migracion + *seed* automaticos).

4.4 Funcionalidades minimas exigidas

1. Formulario de **login** en GET `/login`, procesado por POST `/login`, que verifica credenciales contra la BD usando la funcion equivalente a `password_verify` (BCryptPasswordEncoder.matches en Java o PasswordHasher en .NET).
2. **Gestion de sesion** tras login exitoso; toda ruta distinta de `/login` exige sesion valida; sin sesion, redirige a `/login` con HTTP 302.
3. **Listado** de materias en GET `/materias` (tabla HTML).
4. **Creacion**: GET `/materias/nueva` muestra el formulario y POST `/materias` lo procesa.
5. **Edicion**: GET `/materias/{id}/editar` y POST `/materias/{id}`.
6. **Eliminacion logica**: POST `/materias/{id}/eliminar` pone `activa=false` (nunca borrado fisico).
7. **Logout** en POST `/logout` que invalida la sesion.

5. Guia de desarrollo — pasos para lograr el resultado de aprendizaje

Los siete pasos que siguen son *tecnologicamente neutros* y cubren tanto PHP 8.3 como Spring Boot 3 o ASP.NET Core 8. Cada paso conecta con un contenido de la Unidad II del libro de texto de la asignatura.

Paso 1. Iniciar el proyecto en la pila elegida (Semana 5, 6 o 7)

- **PHP 8.3:** carpeta con `index.php`, `router simple` y `composer.json`; usar `declare(strict_types=1)`; en cada archivo [2].
- **Spring Boot 3:** `start.spring.io` con dependencias *Web*, *Security*, *JPA*, *Validation* y driver de BD; Java 21 LTS [5].
- **ASP.NET Core 8:** `dotnet new mvc -n MateriasApp`, agregar *EF Core*, *Identity* o autenticacion basada en cookies [1].

Paso 2. Modelar la persistencia con SQL parametrizado (Semana 5 y refuerzo Semana 9)

Aplicar el esquema SQL siguiente (adaptable a MySQL, PostgreSQL o SQLite) y **prohibir la concatenacion** de datos del usuario al SQL: usar PDO + marcadores en PHP, `PreparedStatement` o JPA en Java, `DbCommand` o EF Core en .NET [4].

```
CREATE TABLE usuarios (  
  id          BIGSERIAL PRIMARY KEY,  
  username    VARCHAR(50)  NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  rol         VARCHAR(20)  NOT NULL DEFAULT 'ADMIN'  
);  
CREATE TABLE materias (  
  id          BIGSERIAL PRIMARY KEY,  
  codigo      CHAR(6)      NOT NULL UNIQUE,  
  nombre      VARCHAR(80)  NOT NULL,  
  credits     SMALLINT     NOT NULL CHECK (credits BETWEEN 1 AND 6),  
  semestre    SMALLINT     NOT NULL CHECK (semestre BETWEEN 1 AND 10),  
  activa      BOOLEAN      NOT NULL DEFAULT TRUE  
);
```

Paso 3. Implementar autenticacion con hash BCrypt (Semana 9)

Al registrar al usuario semilla, aplicar `password_hash($clave, PASSWORD_DEFAULT)` en PHP, `BCryptPasswordEncoder` en Spring Security, o `PasswordHasher` en ASP.NET Core Identity. **Nunca** guardar contraseñas en texto plano, MD5 ni SHA-1 [3]. En el login, comparar con la funcion *verify* correspondiente, nunca con `==`.

Paso 4. Iniciar sesion segura y proteger rutas (Semana 9)

Tras login exitoso, iniciar la sesion del servidor. La **cookie de sesion** debe emitirse con las banderas `HttpOnly=true`, `SameSite=Strict` (o Lax justificado) y `Secure=true` cuando la evaluacion sea contra HTTPS [4]. Toda ruta distinta de `/login` debe verificar la sesion; si no existe, redirigir con HTTP 302 a `/login`.

Paso 5. Construir el CRUD por capas (Semana 6)

Separar rigurosamente tres responsabilidades:

- **Controlador / Servlet / Controller:** recibe la peticion, valida sesion, delega en el servicio.
- **Servicio:** logica de negocio (unicidad de codigo, validacion de rangos).
- **DAO / Repository:** unico lugar donde hay SQL o consultas JPA/EF; nunca en el controlador [5].

Implementar las cuatro operaciones (crear, listar, editar, eliminacion logica) con vistas HTML del lado del servidor (JSP/Thymeleaf/Razor/PHP puro con `htmlspecialchars`).

Paso 6. Blindar la aplicacion contra el OWASP Top 10:2021 (Semana 9)

- **Anti-XSS:** escapar toda salida dinamica (`htmlspecialchars(..., ENT_QUOTES, 'UTF-8')` en PHP; Thymeleaf y Razor escapan por defecto; JSP EL con `fn:escapeXml`).
- **Anti-CSRF:** cada formulario POST lleva un token secreto (generado con `random_bytes(32)` en PHP; nativo en Spring Security; `@Html.AntiForgeryToken()` en Razor). Verificar con `hash_equals` o equivalente.

- **Cabeceras HTTP:** Strict-Transport-Security: max-age=31536000; includeSubDomains, X-Frame-Options: DENY, X-Content-Type-Options: nosniff y Content-Security-Policy: default-src 'self' [4].
- **Anti-SQLi:** confirmar que ninguna consulta concatena datos del usuario (*grep* del docente al código).

Paso 7. Validar en cliente y servidor

La validación HTML5 nativa (`required`, `minlength`, `maxlength`, `pattern`, `min`, `max`) es *ayuda al usuario*, no defensa: el **servidor** es la fuente de verdad. Revalidar rangos y longitudes en el servidor y devolver HTTP 400 o 422 con mensajes claros al usuario cuando la entrada sea inválida.

6. Guía de publicación en GitHub — criterio de evaluabilidad

El repositorio es la **fuentes de verdad** de la calificación práctica. Debe permitir que el docente *clone*, *arranque* y *verifique* la aplicación sin conocimiento previo del proyecto ni del estudiante. Ese principio de *evaluabilidad* se traduce en la siguiente estructura y contenidos obligatorios.

6.1 Estructura de directorios sugerida

```
materias-<apellido>/
|-- README.md                # obligatorio: ver 6.2
|-- LICENSE                  # licencia MIT o similar
|-- .gitignore
|-- .env.example              # variables sin secretos reales
|-- docker-compose.yml        # obligatorio: ver 6.3
|-- docs/
|   |-- capturas/              # 4-6 capturas de pantalla como respaldo
|   |   |-- 01-login.png
|   |   |-- 02-listado.png
|   |   |-- 03-alta.png
|   |   |-- 04-edicion.png
|   |   |-- 05-eliminacion-logica.png
|   |-- requests.http          # o colección Postman equivalente
|   |-- decisiones.md          # 3-5 decisiones técnicas justificadas
|-- db/
|   |-- schema.sql             # DDL de las tablas
|   |-- seed.sql               # usuario admin con hash BCrypt real
|-- src/                       # código de la pila elegida
```

6.2 Contenido obligatorio del README.md

El README.md debe permitir al docente reproducir el entorno en menos de cinco minutos. Debe contener las siguientes secciones, en este orden:

1. **Datos del estudiante** (apellidos, nombres, cédula, paralelo, fecha del examen).
2. **Pila tecnológica** elegida y versión exacta (p.ej. PHP 8.3.6, Spring Boot 3.3.4, o .NET 8.0.10).
3. **Requisitos previos** (Docker Desktop 4.x o equivalentes locales).
4. **Instrucciones de arranque en un solo comando:** bloque de código que se pueda copiar y pegar tal cual y arrancar el sistema (ver 6.3).
5. **Credenciales del usuario semilla** (admin / Admin*2026), advirtiendo que se usan solo para la evaluación.
6. **URL local** del sistema una vez levantado (p.ej. `http://localhost:8080/login`).
7. **Como probar** el CRUD (referencia a `docs/requests.http` o a la colección Postman).
8. **Como verificar** las defensas de seguridad activadas (comandos `curl -I` para leer las cabeceras).
9. **Hash del último commit** anterior o igual a las 10H45 (7 caracteres, alineado con el PDF del SGA).

6.3 Arranque en un solo comando — docker-compose.yml

El repositorio debe incluir un `docker-compose.yml` que levante **aplicación + base de datos** y aplique automáticamente las migraciones y el `seed`. Comandos que debe ejecutar el docente y que deben funcionar tal cual:

```
git clone https://github.com/<usuario>/materias-<apellido>.git
cd materias-<apellido>
cp .env.example .env
docker compose up -d --build
# esperar 30-60 s y abrir http://localhost:<puerto>/login
```

6.4 Prohibiciones y salvaguardas

- **Prohibido subir secretos reales** al repositorio (contrasenas, tokens, claves privadas). Solo `.env.example` con valores ficticios.
- **Prohibido subir binarios pesados** (JAR o DLL contruidos): agregar `target/` y `bin/` al `.gitignore`.
- **Obligatorio historial Git** con al menos ocho commits del estudiante distribuidos a lo largo del examen; commits descriptivos siguiendo *Conventional Commits* (`feat:`, `fix:`, `docs:`).
- **Obligatorio dejar el repositorio publico** durante toda la ventana de evaluacion (72 horas posteriores a la entrega).

7. Parte practica — Rubrica de evaluacion (60 %)

Cada criterio se califica en uno de cuatro niveles equivalentes al 100 %, 75 %, 50 % o 25 % de su peso, siguiendo la misma escala del Examen de la Unidad I.

Excelente = 100 % Bueno = 75 % Regular = 50 % Insuficiente = 25 %

Criterio (peso)	Excelente	Bueno	Regular	Insuficiente
Autenticacion segura y sesion (10 %)	Login y logout funcionales; contraseñas en BCrypt; cookie de sesion con HttpOnly, SameSite y Secure cuando aplica; rutas protegidas redirigen a /login.	Autenticacion funcional con una bandera de cookie faltante o justificada en README.	Autenticacion funcional pero con hash debil (SHA-256/MD5) o sin proteccion de rutas.	Contraseñas comparadas en texto plano o sin sesion.
Acceso a datos por capas con SQL parametrizado (10 %)	DAO/Repository con todas las consultas usando sentencias preparadas con marcadores; controlador delgado; servicio con reglas de negocio.	Estructura por capas correcta con una consulta con concatenacion menor documentada.	Estructura parcial; SQL en el controlador o mezcla de sentencias preparadas y concatenacion.	Concatenacion de datos del usuario al SQL (vulnerable a SQLi [4]).
CRUD funcional completo sobre Materia (15 %)	Las cuatro operaciones funcionan de extremo a extremo: crear, listar, editar y eliminacion logica; persistencia verificada en BD; validacion de codigo unico.	CRUD operativo con una operacion con fallo menor (p. ej. redireccion incorrecta despues del POST).	Solo dos o tres operaciones funcionan.	Solo una operacion funciona (tipicamente listar) o CRUD sobre otra entidad.
Defensas activas: XSS, CSRF y cabeceras (10 %)	Escape de salida en toda vista; token CSRF verificado en cada POST; HSTS, X-Frame-Options, nosniff y CSP minima presentes y verificables con curl -I [4].	Las tres defensas presentes con alguna cabecera menor faltante.	Solo dos de las tres defensas presentes.	Solo una defensa presente o defensa aparente sin verificacion real en el servidor.
Validacion en cliente y servidor (5 %)	Validacion HTML5 (required, minlength, pattern) mas revalidacion en el servidor con mensajes claros; rechazo con HTTP 400/422.	Validacion en ambos lados con mensajes poco claros.	Validacion solo en un lado (cliente o servidor).	Sin validacion; aceptada datos invalidos.
Calidad de codigo y entrega (10 %)	Repositorio publico, README.md completo (secciones 1–9 de 6.2), docker-compose.yml que arranca todo con un solo comando, al menos 8 commits descriptivos del estudiante, capturas de respaldo en docs/capturas/.	Codigo limpio con README correcto pero commits pobres o docker-compose que exige un ajuste menor.	Codigo desordenado; pocos commits; el sistema arranca solo con intervencion manual.	Sin control de versiones util; el proyecto no arranca en clonacion limpia.
Subtotal parte practica: 60 %				

Penalizaciones y salvaguardas. (1) El uso de librerías generadoras de código asistidas por IA durante el examen anula los puntos de los criterios de *Acceso a datos por capas*, *CRUD funcional* y *Calidad de código y entrega*, por incumplir el carácter individual del examen. (2) La *concatenacion de datos del usuario al SQL* rebaja el criterio de acceso a datos a *Insuficiente* incluso si el CRUD funciona. (3) La *ausencia de repositorio verificable* lleva la parte practica a cero (habilitante). (4) El repositorio Git es la **fuentes de verdad**: capturas y PDF no sustituyen la verificación del código.

8. Calificacion final

Como se obtiene la nota de la unidad. La calificación final de la Unidad II es la suma directa de ambas partes:
Nota final = Nota teorica (40 %) + Nota practica (60 %),
equivalente al 100 % del examen de la unidad. Se aprueba con nota igual o superior a 7.0/10, conforme al reglamento academico UTEQ.

Referencias

- [1] Andrew Lock. *ASP.NET Core in Action*. Manning Publications, 2nd edition, 2021.
- [2] Robin Nixon. *Learning PHP, MySQL and JavaScript: A Step-by-Step Guide to Creating Dynamic Websites*. O'Reilly Media, 6th edition, 2021.
- [3] Jose Manuel Ortega. *Mastering Python for Networking and Security: Leverage the scripts and libraries of Python version 3.7 and beyond to overcome networking and security issues*. Packt Publishing, 2nd edition, 2022.
- [4] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks, 2021.
- [5] Craig Walls. *Spring in Action*. Manning Publications, 6th edition, 2022.

Docente: Dr. Gleiston Ciceron Guerrero Ulloa, Ph.D. | gguerrero@uteq.edu.ec